CODE:

```
def calc_parity_bits(data):
    m = len(data)
    r = 
    n = m + r
    for i in range(0, m):

def encode(d):
    h = [0] * 7
    h[2] = d[0]
    h[4] = d[1]
    h[5] = d[2]
    h[6] = d[3]

    h[0] = h[2] ^ h[4] ^ h[6]
    h[1] = h[2] ^ h[5] ^ h[6]
    h[3] = h[4] ^ h[5] ^ h[6]

    return h.

def detect_and_correct(h):
    c1 = h[0] ^ h[2] ^ h[4] ^ h[6]
    c2 = h[1] ^ h[2] ^ h[5] ^ h[6]
    c3 = h[3] ^ h[4] ^ h[5] ^ h[6]

    error_pos = (c1 * 1) + (c2 * 2) + (c4 * 4)

    if error_pos != 0:
        print("Error detected at bit position:",
              error_pos, "-Correcting...")
        h[error_pos - 1] ^= 1
    else:
        print("NO error detected.")

    return error_pos
```

08/08/2024 EX:06 ERROR CORRECTION AT DATA LINK LAYER.

AIM:

To write a program to implement error detection and correction using HAMMING code concept. Make a test run to input data stream and verify error correction feature.

ALGORITHM:

1. Start the program.
2. Read the no. of 4-bit data blocks (n) formatting the input data stream.
3. For each 4-bit data block D = d_1 d_2 d_3 d_4, perform Hamming encoding:
   (a) Arrange 7-bits with parity bits at position 1, 2, and 4 and data bits at positions 3, 5, 6 and 7.
   (b) Compute the parity bits using even parity:
   $P_1 = d_1$ XOR d_2 XOR d_4 (covers bit positions 1, 3, 5, 7)
   $P_2 = d_1$ XOR d_3 XOR d_4 (covers bit positions 2, 3, 6, 7)
   $P_3 = d_2$ XOR d_3 XOR d_4 (covers bits positions 4, 5, 6, 7)
   (c) Form the 7-bit Hamming code word: P_1 P_2 d_1 P_4 d_2 d_3 d_4.
4. Simulate transmission by introducing a single-bit error at a chosen bit position in the code word.
5. At the receiver, recompute the check bits (syndrome) from the received code word:
   $C_1 = P_1$ XOR d_1 XOR d_2 XOR d_4
   $C_2 = P_2$ XOR d_1 XOR d_3 XOR d_4
   $C_4 = P_4$ XOR d_2 XOR d_3 XOR d_4.
6. Compute Error Position = C_4 C_2 C_1 (binary no. formed from the check bits)
7. If Error Position = 0, no error has occured. Else, the bit at "Error Position" is inverted to correct the error.
8. Extract the original 4 data bits from the corrected 7-bit code word.
9. Repeat steps 3 to 8 for all n blocks in the data stream.
10. Stop the program.

```
def to_str(bits):
    return "".join(str(b) for b in bits)

def main():
    print("=== HAMMING CODE - ERROR DETECTION AND CORRECTION ===\n")
    n = int(input("Enter no. of 4-bit data blocks in the stream: "))
    for b in range(1, n+1):
        print("Block", b)
        d = list(map(int, input("Enter 4 data bits (space separated): ").split()))
        h = encode(d)
        print("Original Data Bits:", to_str(d))
        print("Encoded Hamming Code (7 bits):", to_str(h))
        pos = int(input("Enter bit position to introduce error (1-7, 0=no error): "))
        if 1 <= pos <= 7:
            h[pos-1] ^= 1
            print("Transmitted code with error induced at pos", pos, ":", to_str(h))
        else:
            print("Transmitted Code (no error introduced):", to_str(h))
        print("Recieved code:", to_str(h))
        print("checking for errors...")
        detect_and_correct(h)
        print("corrected code:", to_str(h))

if __name__ == "__main__":
    main()
```

OUTPUT:

```
HAMMING CODE - ERROR DETECTION AND CORRECTION.
Enter the no. of 4-bit data blocks in the stream: 2
Block 1
Enter 4 data bits (space separated): 1 0 1 1
Original Data Bits : 1 0 1 1
Encoding Hamming code (7 bits) : 0 1 1 0 0 1 1
Transmitted code with error induced at pos 3 : 0 1 0 0 1 1
Recieved code : 0 1 0 0 1 1
checking for errors...
Error detected at bit position : 3 - Correcting....
Corrected code : 0 1 1 0 0 1 1
Recovered Data Bits : 1 0 1 1
```

RESULT:

Thus, the Hamming code with error detection and Correction concept has been executed successfully, in python using sample input data stream.